

Úvod do databázových systémů

Kapitola 10
Jazyk SQL (3)

Příkaz SELECT detailněji

- příkaz sestává z 2 až 5 klauzulí (+ ORDER BY)
 - SELECT
 - projekce na sloupce; definice vypočítaných či souhrnných (agregovaných) sloupců
 - FROM
 - seznam tabulek, z nichž čerpáme
 - je-li jich zadáno víc, provede se kartézský součin
 - WHERE
 - selekce řádků z výstupu klauzule FROM
 - GROUP BY
 - přes které atributy se výsledek předchozího seskupí
 - HAVING
 - selekce řádků z výstupu klauzule GROUP BY

Příkaz SELECT detailněji

- klauzule SELECT

SELECT [{ALL | DISTINCT}] výraz

- výraz může obsahovat
 - výčet sloupců nebo hvězdičku (za všechny sloupce)
 - konstantu – ve výsledku bude sloupec plný konstanty
 - agregační funkci
- **DISTINCT** odstraňuje z výsledku duplicitní řádky
 - vliv na agregační funkce, vstupuje do nich méně řádků
- prvek výrazu může být pojmenovaný

SELECT číslo_s AS 'Číslo silnice'

Příkaz SELECT detailněji

- klauzule SELECT

- v PostgreSQL se dá samotný SELECT použít pro jednoduché výpočty nebo volání funkcí

```
SELECT 1+1;
```

```
SELECT power(2, 8);
```

Příkaz SELECT detailněji

- agregační funkce
 - slouží k výpočtu nějaké souhrnné hodnoty, kombinující údaje z více řádků (skupiny řádků)
 - nejčastěji se používají v kombinaci s klauzulí **GROUP BY** (viz dále)
 - není-li klauzule **GROUP BY** přítomna, vstupem agregační funkce jsou všechny řádky dotazu
 - agregační funkce se aplikuje vždy na jeden sloupec
 - k nejčastějším patří určení počtu prvků, součet, průměr, maximum, minimum apod., PostgreSQL zná ještě např. směrodatnou odchylku, rozptyl, ad.

Příkaz SELECT detailněji

- agregační funkce

- pracují na množině řádků s daným sloupcem:

agregační_funkce ([{ALL | DISTINCT}] jméno_sloupce)

- COUNT(col) – počet neprázdných prvků ve sloupci col
- analogicky SUM, MAX, MIN, AVG
- bez zadání DISTINCT počítají s duplicitními řádky
- **pozor na prázdné hodnoty NULL**
 - každá agregační funkce se k hodnotám NULL chová jinak: COUNT(NULL) vrátí nulu (0), SUM(NULL) vrátí NULL

```
SELECT MAX(délka) AS 'Nejdelší'  
FROM Silnice
```

Příkaz SELECT detailněji

- agregační funkce

- bez zadání GROUP BY je nelze za SELECTem kombinovat s dalšími sloupci (viz dále), tj. není možné například:

```
SELECT příjmení, AVG(plat)
FROM osoba;
```

- v uvedeném příkladu totiž agregační funkce vytvoří z celé tabulky **osoba** jediné číslo - průměrný plat všech osob; stroj by tudíž nevěděl, jaké příjmení z mnoha možných má tomuto jedinému číslu přiřadit.
- dotaz tedy nemá smysluplnou sémantiku a jako takový není povolen

Příkaz SELECT detailněji

■ klauzule FROM

FROM seznam_tabulek

- tabulky v seznamu jsou oddělené čárkami
 - je-li v seznamu více než jedna tabulka, do dalšího vyhodnocení vstupuje jejich **kartézský součin**
- místo obyč. seznamu lze za FROM definovat **spojení** tabulek
- místo skutečných tabulek lze (v některých SŘBD) použít i pohledy nebo další vnořený SELECT blok
- prvek seznamu může být pojmenovaný (tzv. alias)
... FROM Most M

Příkaz SELECT detailněji

■ klauzule WHERE

WHERE logická_podmínka_selekce

- ze základní kolekce n-tic, dané výsledkem klauzule **FROM** (tj. jedna tabulka, kart. součin, spojení tabulek) do výsledku postupují jen ty řádky, které splňují podmínku – selekce
 - vyhodnocení podmínky je TRUE, FALSE nebo NULL!
- logické spojky **AND**, **OR**, **NOT**
- např.

```
SELECT číslo_s, délka
```

```
FROM Silnice
```

```
WHERE vlastník = 'kraj Vysočina' AND třída < 3
```

Příkaz SELECT detailněji

■ klauzule WHERE

- pokud alespoň jeden člen porovnání (=, <>, <, >, <=, >=) nabývá hodnoty NULL, pak je výsledek porovnání NULL
- hodnota aritmetického výrazu, kde alespoň jeden člen je NULL, je také NULL
- **pozor** – toto chování může být závislé na SŘBD!
 - v praxi je lepší se NULL hodnotě ve výrazech vyhnout s pomocí operátorů IS NULL, IS NOT NULL a COALESCE

Příkaz SELECT detailněji

■ klauzule WHERE

- vztah mezi dvěma atributy či atributem a hodnotou
 - =, <>, <, >, <=, >=
- interval hodnot
 - výraz1 [NOT] BETWEEN výraz2 AND výraz3
- srovnání řetězců
 - výraz1 [NOT] LIKE maska
 - maska může obsahovat
 - % (libovolný řetězec)
 - _ (libovolný jeden znak)
 - např.

```
SELECT * FROM Silnice  
WHERE vlastník LIKE '%kraj%'
```

Příkaz SELECT detailněji

- klauzule WHERE

- test na (ne)prázdnou hodnotu

- výraz IS [NOT] NULL

- příslušnost prvku do množiny

- výraz1 [NOT] IN výraz2
 - odpovídá zápisu $\text{výraz1} \in \text{výraz2}$ resp. $\text{výraz1} \notin \text{výraz2}$
 - např.

SELECT * FROM Silnice

WHERE vlastník IN ('stát', 'kraj Vysočina', 'Liberecký kraj')

SELECT třída FROM Silnice

WHERE číslo_s NOT IN (SELECT číslo_s FROM Leží_na)

Příkaz SELECT detailněji

■ klauzule WHERE

■ test (ne)prázdnosti množiny

- [NOT] EXISTS poddotaz
- např.

FROM Leží_na L

S.číslo_s = L.číslo_s)

```
SELECT * FROM Silnice S  
WHERE EXISTS (SELECT *
```

WHERE

■ rozšířené kvantifikátory

- výraz1 { = | <> | < | > | <= | >= } { ANY | ALL } poddotaz
 - { aspoň jeden řádek | všechny řádky } poddotazu vyhovují srovnání
- např.

délka FROM Silnice

vlastník = 'stát')

```
SELECT * FROM Silnice  
WHERE délka > ALL (SELECT
```

WHERE

Agregace a seskupování

- agregační funkce

- už víme, že v klauzuli SELECT lze použít různé agregační funkce pro výpočet nějaké souhrnné hodnoty z určeného sloupce
 - COUNT, SUM, MIN, MAX, AVG, ...
 - výsledná tabulka pak bude obsahovat vypočtený sloupec
 - agregační funkce nelze (bez seskupování) v jednom příkazu kombinovat s jinými sloupci tabulky
 - speciální případy
 - jak se vyhodnotí hodnoty NULL?
 - jak se vyhodnotí duplicitní hodnoty?

Agregace a seskupování

- příklady

- najdi počet silnic první třídy

- COUNT(*) počítá počet řádků (i kdyby byly prázdné)

```
SELECT COUNT(*) AS pocet_silnic_1  
FROM Silnice  
WHERE třída = 1;
```

- totéž trochu jinak

- COUNT(s) počítá počet NOT NULL hodnot ve sloupci s
 - COUNT(NULL) = 0

```
SELECT COUNT(číslo_s) AS pocet_silnic_1  
FROM Silnice  
WHERE třída = 1;
```

Agregace a seskupování

■ příklady

- najdi počet *různých* technologií výstavby mostů
 - každou technologii chceme započítat jen jednou, byť je použita na více mostech
 - **DISTINCT** uvnitř agregační funkce

```
SELECT COUNT(DISTINCT technologie) AS  
pocet_techologii  
FROM Most;
```

- průměrná šířka betonových mostu v cm

```
SELECT AVG(šířka) * 100  
FROM Most  
WHERE technologie = 'beton';
```


Agregace a seskupování

- klauzule **GROUP BY (column)**

- n-tice vystoupivší z klauzulí FROM a WHERE se seskupí podle sloupce **column** tak, že za každou skupinu n-tic se stejnými **hodnotami** ve sloupci **column** se do výsledku dostane jedna n-tice
 - v seskupeném sloupci je pak **hodnota** z řádků, z nichž byl „superřádek“ sloučen (byly všechny stejné)
- co ostatní sloupce (kde mohla být obecně v každém původním řádku jiná hodnota)?
 - buď se ve výsledku neobjeví
 - nebo se na jejich hodnoty použije nějaká agregační funkce a vyrobí se jediná hodnota (třeba průměr)
 - zobecnění agreg. funkcí, nevznikne jednořádková tabulka jako předtím, ale tabulka s tolika řádky, kolik dá GROUP BY

Agregace a seskupování

- klauzule GROUP BY (column)
 - za GROUP BY lze uvést více sloupců, pak se seskupuje podle všech možných kombinací
 - máme-li v dotazu tuto klauzuli, co smíme mít za klauzulí SELECT?
 - ty sloupce, které jsou uvedeny za GROUP BY, tj. podle kterých se slučuje
 - agregační funkce nad jinými sloupci, které pro každou skupinu řádků vygenerují jednoznačnou hodnotu
 - **nic jiného** tam být nesmí, i když se nám zdá, že by to třeba fungovalo, protože hodnoty jsou jednoznačné – překladač to nepozná

Agregace a seskupování

- příklady na **GROUP BY**
 - vypiš *pro každého vlastníka* celkovou délku silnic,
 - seskupíme tabulku Silnice podle vlastníka

```
SELECT vlastník, SUM(délka) AS 'Celková délka'  
FROM Silnice  
GROUP BY vlastník;
```

Agregace a seskupování

■ příklady na GROUP BY

číslo s	třída	start	cíl	vlastník	délka	oprava
21	1	D5	Německo	stát	62	12.6.2009
53	1	Znojmo	Pohořelice	stát	38	30.5.2007
230	2	Nepomuk	Bečov nad Teplou	Plzeňský kraj	88	14.2.1999
276	2	Bělá pod Bezdězem	Kněžmost	Liberecký kraj	21	21.8.2003
403	2	Kouty	Telč	kraj Vysočina	36	2.10.2001
602	2	Pelhřimov	Starý Lískovec	kraj Vysočina	108	5.8.2008
0395	3	Kostelec	Cejle	kraj Vysočina	2,5	28.9.1992
1314	3	Štoky	Smrčná	kraj Vysočina	6,3	10.7.2000

vlastník	Celková délka
stát	100
Plzeňský kraj	88
Liberecký kraj	21
kraj Vysočina	152,8

Agregace a seskupování

- příklady na **GROUP BY**

- vypiš pro každého vlastníka celkovou délku silnic a délku jeho nejdelší silnice

```
SELECT vlastník, SUM(délka), MAX(délka)
FROM Silnice
GROUP BY vlastník;
```

- vypiš pro každého vlastníka délku jeho nejdelší silnice a její start

```
SELECT vlastník, MAX(délka), start
FROM Silnice
GROUP BY vlastník;
```

NELZE!

Agregace a seskupování

- klauzule HAVING
 - kritérium pro skupiny

HAVING logická_podmínka_selekce

- následuje za GROUP BY a vybírá pouze ty „superřádky“, které vyhovují podmínce
- analogie klauzule WHERE za FROM

Agregace a seskupování

■ příklad na HAVING

- najdi celkovou délku silnic pro každého vlastníka, zobraz však jen ty, které mají celkovou délku větší nebo rovno 100 km

```
SELECT vlastník, SUM(délka) AS 'Celková délka'  
FROM Silnice  
GROUP BY vlastník  
HAVING SUM(délka) >= 100,
```

v některých DBMS lze použít alias (tj. 'Celková délka')

vlastník	Celková délka
stát	100
Plzeňský kraj	88
Liberecký kraj	21
kraj Vysočina	152,8

vlastník	Celková délka
stát	100
kraj Vysočina	152,8

SELECT se všemi klauzulemi

■ mějme obecný dotaz

SELECT A_1 , agreg_f(A_2) AS 'Name'

FROM R_1 , R_2

WHERE φ

GROUP BY A_1

HAVING ψ

ORDER BY Name

■ sémantika – jak to v principu pracuje

- to neznamená, že takto to dělá SŘBD – optimalizace

1. vyhodnotí se klauzule FROM

- zde kartézský součin R_1 a R_2 , jinak třeba spojení apod.

2. vyhodnotí se podmínka φ za WHERE

3. výsledek se seskupí podle A_1 (GROUP BY)

- vznik „superřádků“

SELECT se všemi klauzulemi

■ mějme obecný dotaz

SELECT A_1 , agreg_f(A_2) AS 'Name'

FROM R_1 , R_2

WHERE φ

□ □ GROUP BY A_1

HAVING ψ

ORDER BY Name

■ sémantika – jak to v principu pracuje

4. vyhodnotí se klauzule SELECT
 - sloupec A_1 a agregovaná hodnota sloupce A_2 , pojmenovanou jako 'Name'
5. vyhodnotí se klauzule HAVING (selekce dle ψ)
 - ze „superřádků“ se vyberou jen ty, splňující podmínku ψ
6. výsledná tabulka se seřadí (ORDER BY)

SELECT se všemi klauzulemi

- v uvedených krocích lze pochopitelně použít další povolené konstrukty:
 - za FROM: definici spojení, vnořené SELECT bloky...
 - za WHERE a HAVING: vnořené SELECT bloky...
 - za SELECT: aritmetika, podmínka (viz dále), funkce...
- příklad
 - vypiš pro každého vlastníka celkovou délku silnic 1. třídy, zobraz jen vlastníky mající tuto délku větší nebo rovnu 100 km, seřaď sestupně podle této délky

```
SELECT vlastník, SUM(délka) AS 'Celková délka 1. třídy'
FROM Silnice
WHERE třída = 1
GROUP BY vlastník
HAVING SUM(délka) >= 100
ORDER by 'Celková délka 1. třídy' DESC;
```

Množinové operace

- příkaz **SELECT** vrací tabulku (někdy i relaci)
 - můžeme tedy na výstupy více příkazů **SELECT** použít množinové operace
 - **UNION**
 - **INTERSECT**
 - **EXCEPT** (v některých SŘBD: **MINUS**)
 - **POZOR:** operandy (tabulky) musí být kompatibilní
 - bez zadání klauzule **ALL** (např. **UNION ALL**) se automaticky eliminují duplicity (tj. výsledek je relace)

Množinové operace

■ příklad

■ najdi čísla silnic, na nichž neleží žádný most

■ postup:

- jsou to takové silnice, jejichž číslo není uvedeno v tabulce Leží_na
- výsledek tedy získáme například tak, že vezmeme seznam čísel všech silnic a od něj množinově odečteme seznam čísel silnic, na nichž leží aspoň jeden most
- to, co zbyde, jsou tudíž čísla silnic bez mostů

```
(SELECT číslo_s FROM Silnice)  
EXCEPT  
(SELECT číslo_s FROM Leží_na);
```

Vnořené poddotazy

■ příklad

- vyber silnice, které mají stejného vlastníka jako silnice č. 403

```
SELECT S1.*  
FROM Silnice S1  
WHERE vlastník =  
    (SELECT vlastník  
     FROM Silnice S2  
     WHERE S2.číslo_s = 403);
```

je bezpečnější použít zde alias

- rovnítko očekává na pravé straně jedinou hodnotu!
 - v tomto případě splněno
 - číslo_s je PK, bude tedy vybrán opravdu je jeden vlastník

Vnořené poddotazy

■ příklad

- vypiš informaci o silnici (...cích) s maximální délkou
 - uvědomte si, že takových silnic může být víc
 - takto to **nejde**:

```
SELECT *  
FROM Silnice  
WHERE délka = MAX(délka);
```

MAX nelze použít mimo SELECT

- správné řešení:

```
SELECT *  
FROM Silnice  
WHERE délka = (SELECT MAX(délka) FROM Silnice);
```

Vnořené poddotazy

- vztažený poddotaz
 - souvisejí nějak s hodnotami řádku zpracovávaného v hlavním dotazu
 - odvolávají se na nadřazený dotaz, jejich vyhodnocení je obvykle náročnější než u dotazů nevztažených
 - lze použít v klauzulích WHERE, FROM a SELECT
 - příklad
 - vyber silnice, na kterých leží právě jeden most

```
SELECT *  
FROM Silnice S  
WHERE (SELECT COUNT(*)  
       FROM Leží_na L  
       WHERE L.číslo_s = S.číslo_s) = 1;
```

Vnořené poddotazy

- vztažený poddotaz

- další příklad

- vyber silnice na Vysočině, na nichž leží víc mostů, než je průměrný počet mostů na silnici na Vysočině
 1. počty mostů na každé silnici na Vysočině
 2. průměr z těchto počtů
 3. počet mostů pro každý řádek porovnat s průměrem

Vnořené poddotazy

```
SELECT *
FROM Silnice S1
WHERE vlastník = 'kraj Vysočina' AND
      (SELECT COUNT(*)
       FROM Leží_na L1
       WHERE L1.číslo_s = S1.číslo_s) >
      (SELECT AVG(PoctyMostu.pocet)
       FROM
         (SELECT číslo_s, COUNT(číslo_m) AS pocet
          FROM Leží_na RIGHT JOIN Silnice
          USING(číslo_s)
          WHERE vlastník = 'kraj Vysočina'
          GROUP BY číslo_s) AS PoctyMostu
       );
```